

Compiler_Project3_Semantic_analysis

2019060546

주원웅

1. 컴파일

```
juwowa@juwowa: ~/loucomp_3
# Makefile for C-Minus
#
# ./lex/tiny.l      --> ./cminus.l (from Project 1)
# ./yacc/tiny.y     --> ./cminus.y (from Project 2)
# ./yacc/globals.h  --> ./globals.h (from Project 2)

CC = gcc

CFLAGS = -W -Wall -g

OBJS = main.o util.o lex.yy.o y.tab.o symtab.o analyze.o

.PHONY: all clean
all: cminus_semantic

clean:
    rm -vf cminus_semantic *.o lex.yy.o y.tab.o y.tab.h y.output

cminus_semantic: $(OBJS)
    $(CC) $(CFLAGS) $(OBJS) -o $@ -lfl

main.o: main.c globals.h util.h scan.h parse.h y.tab.h analyze.h
    $(CC) $(CFLAGS) -c main.c

util.o: util.c util.h globals.h y.tab.h
    $(CC) $(CFLAGS) -c util.c

lex.yy.o: lex.yy.c scan.h globals.h y.tab.h util.h
    $(CC) $(CFLAGS) -c lex.yy.c

lex.yy.c: cminus.l
    flex cminus.l

y.tab.h: y.tab.c
    yacc -b y.tab.c parse.h
    $(CC) $(CFLAGS) -c y.tab.c
```

```
lex.yy.c:1282:17: warning: 'yyunput' defined but not used [-Wunused-function]
1282 | static void yyunput(int c, char *yy_bp)
     | ^~~~~~
gcc -W -Wall -g -c y.tab.c
gcc -W -Wall -g -c symtab.c
symtab.c: In function 'printGlobalSym':
symtab.c:302:20: warning: unused variable 'l' [-Wunused-variable]
302 |     LineList l = b->lines;
     |                ^
gcc -W -Wall -g -c analyze.c
analyze.c:44:8: warning: return type defaults to 'int' [-Wimplicit-int]
44 | static pushBuiltInFunc(void)
   | ^~~~~~
analyze.c: In function 'insertNode':
analyze.c:162:9: warning: implicit declaration of function 'st_print_lineno'; did you mean 'st_add_lineno'? [-Wimplicit-function-declaration]
162 |     st_print_lineno(funcName);
     |     ^~~~~~
analyze.c:155:5: warning: enumeration value 'NonParamDK' not handled in switch [-Wswitch]
155 |     switch (t->kind.dc)
     |     ^~~~~~
analyze.c: In function 'checkNode':
analyze.c:312:33: warning: format '%s' expects argument of type 'char *', but argument 3 has type 'unsigned int' [-Wformat=]
312 |     fprintf(listing, "%s\n", t->child[0]->type);
     |                        ^~      ^~~~~~
     |                        |      |
     |                        char *  unsigned int
analyze.c: In function 'pushBuiltInFunc':
analyze.c:90:1: warning: control reaches end of non-void function [-Wreturn-type]
90 | }
   | ^
At top level:
analyze.c:96:13: warning: 'nullProc' defined but not used [-Wunused-function]
96 | static void nullProc(TreeNode *t)
     | ^~~~~~
gcc -W -Wall -g main.o util.o lex.yy.o y.tab.o symtab.o analyze.o -o cminus_semantic -lfl
juwowa@juwowa: ~/loucomp_3
```

2. 개발 환경

- Ubuntu 22.04

3. 구현과 작동

1. main.c

테스트할 때는 TraceAnalyze 를 TRUE로 세팅했고, 제출 코드는 FALSE 로 설정했다.

2. symtab.c & symtab.h (Print & Manage Symbol Table)

symtab.c 와 symtab.h 는 Symbol Table을 만드는데 사용된다. scope와 bucket을 관리하기 위한 구조체를 symtab.h에 정의했다.

```
typedef struct BucketListRec
{
    char *name;
    TreeNode *treeNode;
    LineList lines;
    int memloc; /* memory location for variable */
    struct BucketListRec *next;
} *BucketList;

typedef struct ScopeListRec
{
    char *scopeName;
    int nestedLevel;
    BucketList bucket[SIZE];
    struct ScopeListRec *parent;
} *ScopeList;
```

scope는 항상 현재의 scope정보를 가지고 있어야 하고, 계층을 이루어야 하기 때문에 스택으로 구현했다.

- Scope 에 대해 create, push, pop할 수 있는 함수를 각각 만들어주었다.

```
void sc_push(ScopeList scope);

void sc_pop(void);

ScopeList sc_top(void);

ScopeList sc_create(char *scopeName);
```

symbol table 출력을 위한 함수들을 추가했다. < Functions >, < Global Symbols >, < Scopes >, < Symbol Table > 출력하는 함수이다.

```
void printFuncTab(FILE *listing);  
  
void printGlobalSym(FILE *listing);  
  
void printScope(FILE *listing);  
  
void printST(FILE *listing);
```

현재 scope를 탐색하는 함수와, scope를 타고 따라 올라가면서 원하는 bucket을 찾는 함수를 추가했다.

```
int st_lookup(char *name);  
  
/* Procedure printSymTab prints a formatted  
 * listing of the symbol table contents  
 * to the listing file  
 */  
  
BucketList st_lookup_bucket(char *name);
```

3 analyze.c & analyze.h (Type Checking & Build Symbol Table)

analyze.c 와 analyze.h 는 AST를 기반으로 Symbol Table을 생성하고, 만들어진 Symbol Table과 AST를 이용해 서 Type Checking을 수행한다.

각 scope마다 Symbol Table에 insert가 완료되면 scope를 pop 해야하므로 function pointer로 사용될 함수를 정의했다.

Type checking을 하기 전 scope를 넣고, function이름과, return 상태를 체크하기 위해 function pointer로 사용될 함수를 정의했다.

```
static void afterInsertNode(TreeNode *t);  
  
static void beforeCheckNode(TreeNode *t);
```

Built-In function인 int input(void)와 void output(int value)를 추가해주기 위한 함수 pushBuiltInFunc 을 정의했다.

각각 타입에 맞는 정보를 할당해주고 symbol table에 insert했다.

output의 경우 value를 parameter로 가지기 때문에 output이라는 scope를 정의한 후 output scope를 집어 넣고, value를 집어넣은 후 scope를 pop했다.

Symbol table를 만드는 작업은 static void insertNode(TreeNode * t) 로 수행했다. Scope 를 만들고 스택에 넣는 작업을 하도록 했다. symbol table을 찾아보며 Redefined error를 잡도록했다.

Type Checking은 static void checkNode(TreeNode * t) 로 수행했다.

4 Test Case & Result

1. Test_1.cm

```
juwowa@juwowa: ~/loucomp_3
juwowa@juwowa:~/loucomp_3$ ./cminus_semantic test_1.cm
C-MINUS COMPILATION: ./test_1.cm
Building Symbol Table...

< Symbol Table >
Symbol Name  Symbol Kind  Symbol Type  Scope Name  Location  Line Numbers
-----
main         Function    void         global      3         11
input        Function    int          global      0         0  14  14
output       Function    void         global      1         0  15
gcd          Function    int          global      2         4  7  15
value        Variable    int          output      0         0
u            Variable    int          gcd         0         4  6  7  7
v            Variable    int          gcd         1         4  6  7  7  7
x            Variable    int          main        0         13  14  15
y            Variable    int          main        1         13  14  15

< Functions >
Function Name  Return Type  Parameter Name  Parameter Type
-----
main           void
input          int
output         void
-              -            value          int
gcd            int
-              -            u              int
-              -            v              int

< Global Symbols >
Symbol Name  Symbol Kind  Symbol Type
-----
main         Function    void
input        Function    int
output       Function    void
gcd          Function    int

< Scopes >
Scope Name  Nested Level  Symbol Name  Symbol Type
-----
output      1            value       int

gcd         1            u           int
gcd         1            v           int

main        1            x           int
main        1            y           int

Checking Types...
Type Checking Finished
```

2. Test_2.cm

```
juwowa@juwowa: ~/loucomp_3
Type Checking Finished
juwowa@juwowa:~/loucomp_3$ ./cminus_semantic test_2.cm

C-MINUS COMPILATION: ./test_2.cm

Building Symbol Table...

< Symbol Table >
Symbol Name  Symbol Kind  Symbol Type  Scope Name  Location  Line Numbers
-----
main         Function    void        global      2         1
input        Function    int         global      0         0  8
output       Function    void        global      1         0  18
value        Variable    int         output      0         0
i            Variable    int         main        0         3  5  6  8  10  10  13  14  16  18
x            Variable    int[]        main        1         3  8  16  18

< Functions >
Function Name  Return Type  Parameter Name  Parameter Type
-----
main          void
input         int
output        void
-             -            value          int

< Global Symbols >
Symbol Name  Symbol Kind  Symbol Type
-----
main         Function    void
input        Function    int
output       Function    void

< Scopes >
Scope Name  Nested Level  Symbol Name  Symbol Type
-----
output      1            value       int
main        1            i           int
main        1            x           int[]

Checking Types...

Type Checking Finished
juwowa@juwowa:~/loucomp_3$
```

3. Test_3.cm

```
juwowa@juwowa: ~/loucomp_3
juwowa@juwowa:~/loucomp_3$ ./cminus_semantic test_3.cm
C-MINUS COMPILATION: ./test_3.cm
Building Symbol Table...

< Symbol Table >
Symbol Name Symbol Kind Symbol Type Scope Name Location Line Numbers
-----
main         Function   int      global    3         6
input        Function   int      global    0         0
output       Function   void     global    1         0
x            Function   int      global    2         1 12
value        Variable   int      output    0         0
y            Variable   int      x         0         1 3
a            Variable   int      main      0         8 12
b            Variable   int      main      1         9 12
c            Variable   int      main      2         10 12

< Functions >
Function Name Return Type Parameter Name Parameter Type
-----
main         int      -             void
input        int      -             void
output       void     -             -
-            -        value         int
x            int      -             y         int
-            -        -             -

< Global Symbols >
Symbol Name Symbol Kind Symbol Type
-----
main         Function   int
input        Function   int
output       Function   void
x            Function   int

< Scopes >
Scope Name Nested Level Symbol Name Symbol Type
-----
output      1          value      int
x           1          y          int
main        1          a          int
main        1          b          int
main        1          c          int

Checking Types...
Error: Invalid function call at line 12 (name : "x")

Type Checking Finished
11% 11/28, 4:33 PM
```

4. Test_4.cm

```
juwowa@juwowa: ~/loucomp_3
juwowa@juwowa:~/loucomp_3$ ./cminus_semantic test_4.cm
C-MINUS COMPILATION: ./test_4.cm
Building Symbol Table...

< Symbol Table >
Symbol Name  Symbol Kind  Symbol Type  Scope Name  Location  Line Numbers
-----
main         Function    int         global     2         1
input        Function    int         global     0         0
output       Function    void        global     1         0   4
value        Variable    int         output     0         0
x            Variable    int[]       main       0         3   4

< Functions >
Function Name  Return Type  Parameter Name  Parameter Type
-----
main          int
input         int
output        void
-             -            value          int

< Global Symbols >
Symbol Name  Symbol Kind  Symbol Type
-----
main         Function    int
input        Function    int
output       Function    void

< Scopes >
Scope Name  Nested Level  Symbol Name  Symbol Type
-----
output      1            value       int
main        1            x           int[]

Checking Types...
Error: Invalid array indexing at line 4 (name : "x"). indices should be integer

Type Checking Finished
juwowa@juwowa:~/loucomp_3$
```